

Задача А. Математический турнир

Если все числа различные, то нам надо брать самые маленькие из них, пока сумма не превышает m . Для этого достаточно их отсортировать в возрастающем порядке и взять несколько первых.

Для полного решения задачи сначала надо избавиться от одинаковых чисел. Это можно сделать либо путем добавления их в контейнер, аналогичный `std::set` в C++, либо отсортировать и удалить одинаковые соседние (можно при этом воспользоваться алгоритмом STL `std::unique`). Общая сложность такого решения будет $O(n \cdot \ln(n))$.

Задача В. Равенство

Для каждого количества медалей k от 0 до 6 нужно посчитать, сколько команд можно составить с таким количеством медалей.

Для $k = 0$ понятно, что мы можем составить лишь $\frac{c_0}{3}$ команд.

Для $k = 1$ мы должны взять в команду одного человека с одной медалью и двух — без медалей. Поэтому ответ — минимальное значение из c_1 и $\frac{c_0}{2}$.

При $k = 2$ можно брать либо двух людей с одной медалью, либо одного человека с двумя. Первый вариант выгоднее, поскольку он требует добавления всего лишь одного человека без медалей вместо двух. Поэтому сначала надо взять $\min(\frac{c_1}{2}, c_0)$ команд первого типа, а затем из оставшихся школьников составить еще $\min(\frac{c_0}{2}, c_2)$ команд.

Для $k = 3$ есть тоже два варианта: $0 + 1 + 2$ медалей или $1 + 1 + 1$ медалей. Аналогично, первый вариант выгоднее, поскольку задействует только одного человека с одной медалью. Таких команд можно составить $\min(c_0, c_1, c_2)$. Если после этого остались еще ученики с одной медалью, из них можно дополнительно составить $\frac{c_1}{3}$ команд.

Аналогичные рассуждения можно привести и для случаев $k = 4, 5, 6$. Однако, можно заметить, что при $k > 3$ мы можем заменить человека с x медалями на человека с $2 - x$ медалями и получить аналогичную задачу для $6 - k < 3$. Для этого достаточно поменять друг с другом c_0 и c_2 и найти количество команд для $k' = 6 - k$ описанными выше способами. Таким образом, можно сократить количество рассматриваемых случаев почти в 2 раза.

Задача С. Лена и лестница

Основная идея задачи — использование динамического программирования. Действительно, можно заметить, что если мы знаем, где находится «верхняя» нога, то нам неважно, на какой именно ступеньке находится другая (важно только, на этой же или ниже), поскольку это не влияет на варианты движения этой ногой. Тогда можно посчитать значения $dp[pos][state]$ — минимальное количество шагов, которое еще необходимо сделать, если сейчас верхняя нога находится на ступеньке pos , а $state$ описывает множество ног на этой ступеньке (левая, правая или обе). Тогда можно перебрать, какой ногой сделать следующий шаг и на какую ступеньку поставить ногу.

Если ставить одну ногу выше другой, то надо перебрать, на какую ступеньку эту ногу поставить (пока позволяет длина шага). Таким образом, получится переход $dp[pos][state] = 1 + dp[npos][nstate]$, где $npos$ — новая позиция «верхней» ноги, а $nstate$ описывает новое состояние. Получится решение с вычислительной сложностью $O(n^2)$.

Для ускорения решения достаточно заметить, что вместо перебора позиции $npos$ можно ограничиться максимальным достижимым значением (ответ в этом случае будет минимальным). Найти это значение можно при помощи бинарного поиска по массиву префиксных сумм высот ступенек. Тогда общая сложность будет $O(n \cdot \ln(n))$, что достаточно для полного решения. Такой подход можно улучшить при помощи метода двух указателей, чтобы избавиться от бинарного поиска, однако, такое ускорение необязательно.

Задача D. Фотографии со сборов

Можно переформулировать задачу следующим образом — найти количество способов составить таблицу из k строк и n столбцов, где строки соответствуют фотографиям, а столбцы — людям. Будем записывать в ячейку таблицы 0, если соответствующий человек отсутствует на фотографии и 1, если присутствует. Тогда должно выполняться, что в каждой строке есть хотя бы одна единица, а в каждом столбце количество единиц одинаково.

Для решения первой подгруппы достаточно перебрать все возможные варианты заполнения, коих 2^{nm} , и проверить каждый на соответствие условию выше.

Для второй подгруппы можно использовать динамическое программирование по профилю (поддерживать маску строк без единиц, количество единиц в текущем столбце и количество единиц в предыдущем).

Для полного решения можно действовать следующим образом. Будем перебирать значение t — количество единиц в каждом столбце ($1 \leq t \leq k$). Тогда для расстановки t единиц существует $C(k, t)$ вариантов (где C — число сочетаний). Если расставлять варианты для столбцов независимо, то мы получим $C(k, t)^n$ вариантов для заданного t и $f(n) = \sum_{t=1}^k C(k, t)^n$ всего. Эти значения можно найти при помощи предподсчета числа сочетаний и бинарного возведения в степень за $O(k \cdot n \cdot \ln(n))$ в сумме. Однако, при таком решении мы учтём и варианты, где есть пустые строки (без единиц). Поэтому для каждого $m < n$ нужно вычесть из ответа количество вариантов составить таблицу, где m строк пустые. Но вариант с m пустыми строками соответствует исходной задаче для $n - m$ строк, только это значение нужно домножить на количество вариантов выбрать m пустых строк из n исходных, то есть, на значение $C(n, m)$. Итого, получаем, что ответ на задачу для n строк равен $ans(n) = f(n) - \sum_{m=1}^n C(n, m) \cdot ans(n - m)$.

Задача Е. Деревья

Поскольку домик можно поворачивать, то для удобства можно рассматривать лишь случаи, где первая сторона домика повернута относительно стороны n участка менее, чем на 90 градусов. При этом нужно найти ответ для домика $w \times h$, а при $w \neq h$ добавить ответ для домика $h \times w$.

В первой подзадаче можно заметить, что домик может быть расположен только параллельно сторонам участка. Количество способов выбрать левый-верхний угол домика, чтобы правый-нижний угол не выходил за пределы участка, равно $(n - w + 1) \cdot (m - h + 1)$.

Во второй подзадаче можно перебрать все позиции трёх вершин домика, при этом четвертая будет определяться однозначно. Для каждого варианта надо проверить, что длины сторон домика соответствуют требуемым и что соседние стороны ортогональны. Вычислительная сложность решения будет $O((n \cdot m)^3)$.

Для полного решения нужно учесть способы расположить домик параллельно сторонам участка (как в первой подзадаче) и способы расположить домик под углом. Во втором случае стороны домика будут являться гипотенузами подобных прямоугольных треугольников, катеты которых расположены параллельно сторонам участка. Поскольку w и h являются гипотенузами подобных треугольников, то существует подобный им треугольник (катеты которого также расположены параллельно сторонам участка) с гипотенузой $d = \gcd(w, h)$, где $\gcd$ — наибольший общий делитель. Тогда можно перебрать длину первого катета $x < d$ и проверить, что длина второго катета y получится целочисленной, используя теорему Пифагора. Если значение y получилось целочисленным, то легко можно найти высоту и ширину домика относительно координатных осей:

$$W = x \cdot (w/d) + y \cdot (h/d)$$

$$H = y \cdot (w/d) + x \cdot (h/d)$$

Далее для подсчета количества вариантов расположить домик с таким поворотом нужно еще раз воспользоваться формулой из первой подзадачи для значений W и H . Для каждого x ответ находится за $O(1)$, значит, общая сложность решения будет $O(h + w)$.

Задача Ф. Ограбление каравана

Условие задачи можно переформулировать следующим образом — дан взвешенный неориентированный граф и есть q запросов. Каждый запрос описывается парой вершин и нужно сообщить, можно ли удалить из графа одно ребро, чтобы между этими вершинами не существовало пути. Среди таких рёбер нужно найти ребро наименьшего веса.

Для решения первой подзадачи достаточно для каждого запроса перебрать все рёбра. Для каждого ребра нужно удалить его из графа и проверить, существует ли путь между вершинами запроса — это можно сделать поиском в ширину или в глубину за $O(m)$ операций. Общая сложность такого решения есть $O(q \cdot m^2)$.

Для второй подзадачи сначала для каждого ребра нужно разбить граф на компоненты связности при удалении этого ребра — то есть для каждой вершины получить номер компоненты. Это можно сделать за $O(m^2)$. Далее для каждого запроса нужно перебрать удаляемое ребро и проверить, находятся ли вершины запроса в разных компонентах связности за $O(1)$. Итого, общая сложность решения $O(m^2 + q \cdot m)$.

В третьей подзадаче граф является деревом, поэтому достаточно найти минимальное по весу ребро на пути между вершинами запроса. Для этого сначала подвесим дерево за произвольную вершину и предсчитаем для каждой вершины предков и минимальные веса рёбер на пути длины 2^k методом двоичного подъёма. Далее для каждого запроса можно будет находить наименьшее ребро стандартным алгоритмом нахождения LCA (наименьшего общего предка) за $O(\ln(n))$ на каждый запрос. Общая сложность алгоритма $O(n \cdot \ln(n) + q \cdot \ln(n))$.

Для полного решения задачи нужно сначала найти мосты в графе за $O(m + n)$ операций. Далее нужно разбить граф на компоненты двусвязности и построить деревья этих компонент. Затем для каждого запроса нужно проверить, находятся ли вершины запроса в одной компоненте двусвязности. Если они находятся в одной компоненте, то ответ -1 (нельзя удалить ребро, чтобы не существовало пути между вершинами), в противном случае нужно найти ответ, как в предыдущей подзадаче. Общая сложность решения $O(m + n \cdot \ln(n) + q \cdot \ln(n))$.

Задача G. Мощные пары

В первой подзадаче достаточно перебрать все пары чисел и проверить, является ли их произведение степенью натурального числа. Максимальная возможная степень при этом может быть $M = 2 \cdot \log_2(x_i)$. Итого, сложность решения $O(n^2 \cdot M)$.

Для остальных подзадач сначала будем считать, что у нас есть разложение каждого числа на простые множители, а именно будем хранить список простых делителей и их степеней. Максимальная длина списка также будет $O(M)$. Затем переберем значение k — какую степень натурального числа мы хотим получить в произведении, значение k может быть в пределах от 2 до M . Для каждого k будем решать задачу независимо — будем искать количество пар, таких что $x_i \cdot x_j = a^k$ для некоторого a . При этом некоторые пары мы учтем несколько раз, например, если произведение чисел равно 64, то мы один раз учтем его как 2^6 , один раз как 4^3 и один раз как 8^2 . Для устранения этой проблемы следует воспользоваться стандартным принципом включений и исключений для значений k (см. также функцию Мёбиуса).

Для фиксированного k будем проходить по всем числам x_i исходного массива и добавлять в словарь (например, `std::map` в C++) список простых чисел разложения x_i . При этом от степеней будем оставлять только остаток при делении на k . При этом при рассмотрении очередного числа легко найти, каким должен быть список второго числа в паре, чтобы в произведении каждая степень простого делилась на k . А именно, для каждой степени $t > 0$ простого числа в списке текущего числа, это же простое число должно иметь степень $k - t$ в списке парного ему числа. Отметим, что мы получим единственный подходящий список (поскольку все степени взяты по модулю k и имеют значения от 1 до $k - 1$). Тогда нужно взять количество таких списков из словаря, что можно сделать за $O(M \cdot \ln(n))$ для каждого числа. Для ускорения работы можно заранее проверить, что требуемый список соответствует слишком большому числу (например, при $k \geq M/2$ мы не можем получить больше двух простых чисел в произведении), чтобы избавиться от обращения к словарю. Кроме того, отдельно нужно обработать случаи при $x_i = 1$, поскольку каждая пара единиц в таком алгоритме увеличивает ответ, однако, она не является мощной.

Наконец, в зависимости от подзадачи, нужно разложить исходные числа на простые множители. Для второй подзадачи достаточно разложить каждое число отдельно за $O(\sqrt{x_i})$.

Для третьей подзадачи заметим, что достаточно перебирать делители до $\sqrt[k]{x_i}$. После удаления таких делителей из числа у нас может остаться величина y_i — либо квадрат простого числа, либо простое число, либо произведение двух различных простых. Первый случай легко проверить за $O(1)$. Заметим, что второй и третий случаи для данной задачи одинаковы, поскольку для нахождения мощной пары данного числа потребуется делитель y_i^{k-1} у второго числа в паре. При этом важно перебирать простые делители до некоторой границы $d = \sqrt[k]{\max(x_i)}$, одинаковой для всех чисел, иначе одно и то же значение y_i может в некоторых числах быть разложенным на произведение двух

простых, а в других — остаться одним числом. Итого, сложность алгоритма $O(n\sqrt[3]{\max(x_i)})$.

Для полного решения нужно применить алгоритм из предыдущей подзадачи, но перебирать только простые делители, меньшие d , что позволит ускорить решение в $\ln(d)$ раз.